

Multi-Layer Perceptron for Handwritten Digit Classification: A From-Scratch and PyTorch Comparison

Parth Mishra¹ and Rocky Sehwat¹

Department of Electronics and Communication Engineering
Bharati Vidyapeeth (Deemed to be University), Pune, India
PRN: 2414110674 (Parth Mishra); 2414110684 (Rocky Sehwat)

Abstract. This paper presents the implementation and evaluation of a Multi-Layer Perceptron (MLP) for handwritten digit classification using the MNIST dataset. The network was implemented from scratch using NumPy to understand forward propagation, activation functions, Softmax, cross-entropy loss, backpropagation, and parameter updates. ReLU and Sigmoid activation functions were compared experimentally. The same architecture was then implemented using the PyTorch deep learning framework. The network consists of 784 input features, two hidden layers containing 128 and 64 neurons, and an output layer with 10 classes. The from-scratch ReLU model achieved 96.07% test accuracy, while the Sigmoid model achieved 94.88%. The PyTorch implementation using ReLU achieved 96.77% test accuracy. The results illustrate the effect of activation functions in the tested models and demonstrate how a deep learning framework simplifies neural network implementation.

Keywords: Multi-Layer Perceptron · MNIST · Neural Networks · ReLU · Sigmoid · Backpropagation · PyTorch

1 Introduction

Handwritten digit recognition is a classification problem in which an image is assigned to one of ten classes, representing digits 0 through 9. It is widely studied in machine learning and computer vision. Neural networks can learn patterns from labelled image data and use them to classify previously unseen images [3, 4].

A Multi-Layer Perceptron (MLP) is a feed-forward neural network containing an input layer, hidden layers, and an output layer. Its weights and biases are adjusted during training to reduce prediction error. Backpropagation calculates gradients that are used to update these parameters [2].

The objective of this work is to implement an MLP from scratch using NumPy, compare ReLU and Sigmoid activation functions, and evaluate an equivalent architecture implemented using PyTorch [8].

2 Dataset and Preprocessing

The MNIST dataset contains 70,000 grayscale images of handwritten digits. Each image has a resolution of 28×28 pixels. The standard dataset contains 60,000 training images and 10,000 test images [1].

Each image was flattened into a vector of 784 features because $28 \times 28 = 784$. Pixel values in the range 0–255 were normalized to the range $[0, 1]$ by dividing by 255. The data was loaded using the scikit-learn OpenML interface, and the experiment used a fixed random state when splitting the data [9].

3 Network Architecture

The MLP architecture is

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10.$$

The input layer represents the image pixels. The hidden layers contain 128 and 64 neurons, while the output layer represents the ten possible digit classes.

For each layer, the weighted input and activation are

$$Z = XW + b,$$

$$A = f(Z),$$

where X is the input, W is the weight matrix, b is the bias vector, and f is the activation function.

3.1 Activation Functions

ReLU (Rectified Linear Unit) is defined as

$$f(x) = \max(0, x).$$

It returns zero for negative inputs and preserves positive inputs. The Sigmoid function is

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Sigmoid maps values to the interval between zero and one. Activation functions introduce non-linearity, enabling neural networks to learn complex patterns [6, 7].

3.2 Output and Loss

Softmax converts the output scores into probabilities for the ten digit classes:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}.$$

The predicted class is the class with the highest probability. Cross-entropy loss measures the difference between the predicted probabilities and the correct labels:

$$L = - \sum_{i=1}^{10} y_i \log(p_i),$$

where y_i is the target label in one-hot form and p_i is the predicted probability [4].

4 Implementation Methodology

4.1 From-Scratch Implementation

The first MLP was implemented using NumPy without a dedicated deep learning framework. The implementation performed forward propagation, applied activation functions, calculated Softmax probabilities and cross-entropy loss, and used backpropagation to calculate gradients.

Weights and biases were updated using gradient descent with a learning rate of 0.1. The models were trained for 10 epochs with a mini-batch size of 128. ReLU and Sigmoid models were trained separately and evaluated on the test set.

4.2 Pseudocode for the Scratch MLP

The following pseudocode summarizes the principal training steps.

Algorithm 1: Train MLP from scratch

Input: Training data X, labels y, epochs E,
batch size B, learning rate eta

1. Initialize weights and biases
2. For each epoch from 1 to E:
 3. Shuffle the training samples
 4. Divide data into mini-batches
 5. For each mini-batch:
 6. Perform forward propagation
 7. Apply hidden-layer activation
 8. Calculate Softmax probabilities

9. Calculate cross-entropy loss
10. Calculate output-layer gradients
11. Backpropagate gradients through
the hidden layers
12. Update weights and biases
13. Return the trained model

Forward propagation produces predictions. Backpropagation uses the chain rule to calculate gradients, and gradient descent updates the parameters to reduce loss [2, 5].

4.3 PyTorch Implementation

The second model used PyTorch, which provides neural network layers, automatic differentiation, loss functions, and optimizers [8]. It used the same architecture: 784 input features, hidden layers of 128 and 64 neurons, and 10 output classes.

ReLU was used between hidden layers. Cross-entropy loss and stochastic gradient descent (SGD) with a learning rate of 0.1 were used for training. The model was trained for 10 epochs using mini-batches of 128 samples.

4.4 Pseudocode for PyTorch

Algorithm 2: Train MLP using PyTorch

1. Define the network layers
2. Initialize the loss function and optimizer
3. For each epoch:
4. Set model to training mode
5. For each mini-batch:
6. Clear previous gradients
7. Calculate model predictions
8. Calculate cross-entropy loss
9. Compute gradients automatically
10. Update parameters using optimizer
11. Evaluate model on the test set
12. Calculate test accuracy and test loss

Unlike the scratch implementation, PyTorch automatically calculates gradients through its automatic differentiation system. The optimizer then updates the parameters.

5 Results and Discussion

Table 1 presents the test performance of the three implementations.

Table 1. Test performance comparison of the MLP models.

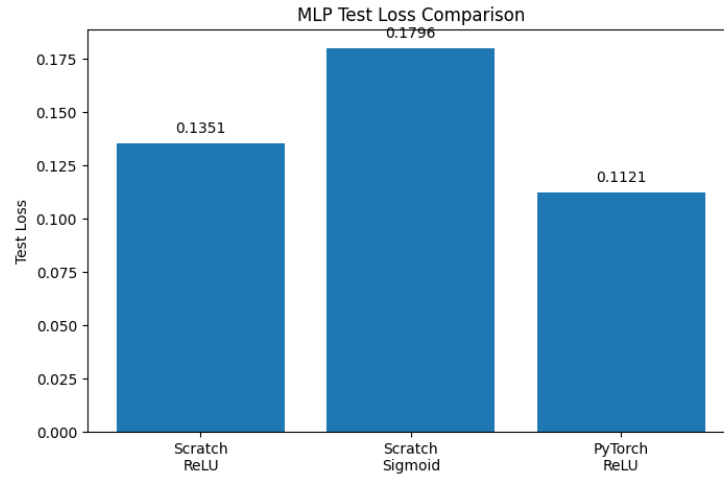
Model	Test Accuracy	Test Loss
Scratch MLP (ReLU)	96.07%	0.1351
Scratch MLP (Sigmoid)	94.88%	0.1796
PyTorch MLP (ReLU)	96.77%	0.1121

The scratch ReLU model achieved higher test accuracy and lower test loss than the scratch Sigmoid model. This indicates that ReLU performed better under the experimental settings used. The PyTorch implementation achieved the highest test accuracy of 96.77% and the lowest test loss of 0.1121.

These results describe the observed experiments and do not establish that PyTorch will always outperform a scratch implementation.

5.1 Accuracy and Loss Graphs

The experimental graphs are included below when the corresponding image files are available in the Overleaf project.

**Fig. 1.** Test accuracy comparison of the three MLP models.

6 Limitations and Future Work

The experiment was limited to one dataset, one main architecture, and a fixed training duration. Results may vary with different initialization methods, hy-

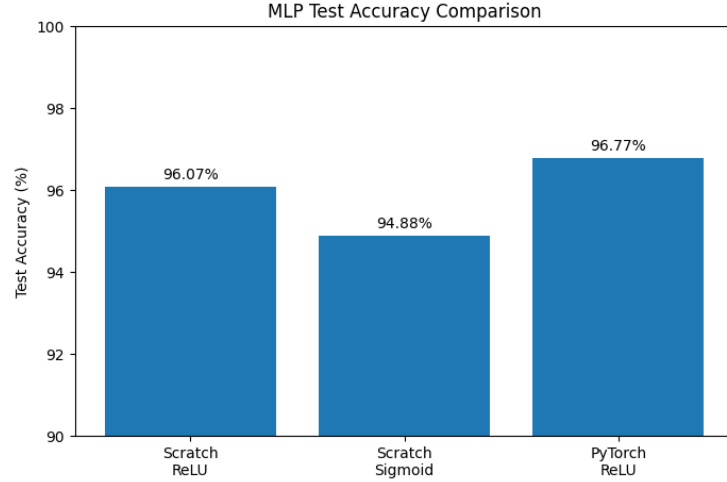


Fig. 2. Test loss comparison of the three MLP models.

perparameters, data splits, and optimization procedures. Future work could investigate additional activation functions, optimizers, and convolutional neural networks for image classification [5].

7 Conclusion

This work implemented an MLP for handwritten digit classification using NumPy and PyTorch. The scratch implementation demonstrated forward propagation, activation functions, loss calculation, backpropagation, and parameter updates. ReLU performed better than Sigmoid in the tested scratch models. The PyTorch model achieved the highest test accuracy of 96.77%. The experiment illustrates the fundamental operations of neural network training and the practical benefits of using a deep learning framework.

References

1. LeCun, Y., Cortes, C., Burges, C.J.: The MNIST database of handwritten digits (1998). <http://yann.lecun.com/exdb/mnist/>
2. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* 323, 533–536 (1986)
3. Bishop, C.M.: *Pattern Recognition and Machine Learning*. Springer, New York (2006)
4. Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. MIT Press, Cambridge (2016)
5. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* 521, 436–444 (2015)

6. Nair, V., Hinton, G.E.: Rectified linear units improve restricted Boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning, pp. 807–814 (2010)
7. Glorot, X., Bordes, A., Bengio, Y.: Deep sparse rectifier neural networks. In: Proceedings of the 14th International Conference on Artificial Intelligence and Statistics, pp. 315–323 (2011)
8. Paszke, A., et al.: PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems 32, pp. 8024–8035 (2019)
9. Pedregosa, F., et al.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12, 2825–2830 (2011)
10. Hastie, T., Tibshirani, R., Friedman, J.: *The Elements of Statistical Learning*, 2nd edn. Springer, New York (2009)
11. Rosenblatt, F.: The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review* 65(6), 386–408 (1958)
12. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: International Conference on Learning Representations (2015)